---

1 Pipes (Anonymous & Named)

◇ Concepts & Working

Pipes provide **unidirectional** communication between processes, meaning one process writes while the other reads.
There are **two types** of pipes:

1. **Anonymous Pipes**: Used **only** between parent-child processes created with `fork()`.
2. **Named Pipes (FIFO)**: Can be used **between any two processes**.

◇ System V Pipes (Anonymous)

- Implemented using `pipe()`.
- Creates **two file descriptors**: `fd[0]` (read) and `fd[1]` (write).
- Only exists as long as the processes using them are alive.

◇ *Example: Anonymous Pipe (System V)*

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2]; // File descriptors for pipe
    pipe(fd);  // Create pipe

    pid_t pid = fork();
    if (pid == 0) {  // Child process
        close(fd[1]); // Close write end
        char buffer[100];
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    } else {  // Parent process
        close(fd[0]); // Close read end
        write(fd[1], "Hello Child!", 12);
        close(fd[1]);
    }
    return 0;
}
```

◆ **Explanation:** 1. `pipe(fd)` creates a pipe with two ends (`fd[0]` for reading, `fd[1]` for writing).

2. The **parent writes** into `fd[1]`, and the **child reads** from `fd[0]`.
3. Pipes are closed after use to prevent **resource leaks**.

🔹 **Limitations:** - Only works between parent-child processes. - Can **only send raw data**, no structure (e.g., numbers need conversion to strings).

◇ POSIX Named Pipes (FIFO)

- Created using `mkfifo()`.
- Behaves like a file and allows **any two processes** to communicate.

◇ *Example: Named Pipe (POSIX FIFO)*

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>

#define FIFO_PATH "/tmp/my_fifo"

int main() {
    mkfifo(FIFO_PATH, 0666);  // Create named pipe

    pid_t pid = fork();
    if (pid == 0) {  // Child Process
        int fd = open(FIFO_PATH, O_RDONLY);
        char buffer[100];
        read(fd, buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd);
    } else {  // Parent Process
        int fd = open(FIFO_PATH, O_WRONLY);
        write(fd, "Hello FIFO!", 11);
        close(fd);
    }
    return 0;
}
```

🔹 **Explanation:** 1. `mkfifo(FIFO_PATH, 0666)` creates a named pipe at `/tmp/my_fifo`.
2. **Unrelated processes** can communicate using this FIFO.
3. Data remains until read by a process.

🔹 **Use Cases:** - Logging servers
- Shell communication (`ls | grep .c`)

---

2 Message Queues

◇ Concepts & Working

- Message queues allow **structured message passing** between processes.
- Data is stored in the queue until **explicitly retrieved**.
- **POSIX** message queues use `mq_open()`.
- **System V** message queues use `msgget()`.

◇ System V Message Queue

Example Code

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg {
    long type;
    char text[100];
};

int main() {
    key_t key = ftok("queuefile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg message;

    message.type = 1;
    strcpy(message.text, "Hello System V IPC!");

    msgsnd(msgid, &message, sizeof(message), 0);
    printf("Message Sent\n");

    return 0;
}
```

◆ **Explanation:** 1. `msgget()` creates a message queue.
2. `msgsnd()` sends a structured message.
3. Messages persist **until deleted manually**.

◆ **Limitations:** - Harder to manage compared to POSIX queues.

◇ POSIX Message Queue

Example Code

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

#define MQ_NAME "/posix_mq"

int main() {
    mqd_t mq = mq_open(MQ_NAME, O_CREAT | O_WRONLY, 0644, NULL);
    mq_send(mq, "Hello POSIX IPC!", 16, 0);
    mq_close(mq);
    return 0;
}
```

🔹 **Explanation:** 1. `mq_open()` creates a message queue with a **file-like** interface.
2. Messages are automatically **deleted** when read.
3. Easier and **more efficient** than System V queues.

🔹 **Performance Comparison** | Feature | System V | POSIX | |———-|———|——-| | **Persistence** | Requires manual deletion | Auto-cleanup | | **Ease of Use** | Complex | Simpler |

3️⃣ Shared Memory

◇ Concepts & Working

- **Fastest** IPC mechanism (direct memory access).
- Used when **high-speed data sharing** is needed.
- **System V** uses `shmget()`, **POSIX** uses `shm_open()`.

◇ POSIX Shared Memory

Example Code

```c
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

#define SHM_NAME "/posix_shm"
```

```c
int main() {
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, 1024);
    char *ptr = mmap(NULL, 1024, PROT_WRITE, MAP_SHARED, fd, 0);

    strcpy(ptr, "POSIX Shared Memory Example");
    printf("Data written: %s\n", ptr);

    munmap(ptr, 1024);
    close(fd);
    return 0;
}
```

🔹 **Use Cases:** - Real-time applications like **video processing**.
- **Database caching** for performance improvement.

Semaphores

◇ Concepts & Working

- Used for **synchronization** between processes.
- Prevents **race conditions** in shared resources.

◇ POSIX Semaphore

Example Code

```c
#include <stdio.h>
#include <fcntl.h>
#include <semaphore.h>

#define SEM_NAME "/posix_sem"

int main() {
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    printf("POSIX Semaphore created\n");
    sem_close(sem);
    return 0;
}
```

🔹 **Use Cases:** - Preventing **race conditions** in critical sections.

Conclusion

IPC Type	System V	POSIX

IPC Type	System V	POSIX
Pipes	Parent-child	Any process
Message Queues	Complex	Simpler
Shared Memory	Manual cleanup	Auto-managed
Semaphores	Harder to use	Easier